

ИСПОЛЬЗОВАНИЕ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА В ПОИСКЕ УЯЗВИМОСТЕЙ КОДА И СРАВНИТЕЛЬНЫЙ АНАЛИЗ РАЗЛИЧНЫХ МЕТОДОВ ТЕСТИРОВАНИЯ

А.П. Захаров, М.А. Маслова

В статье представлен аналитический обзор возможностей применения технологий искусственного интеллекта для обнаружения критических уязвимостей, входящих в список MITRE CWE Top 25 2024 года. Исследование содержит синтез и критическое сопоставление литературных данных по эффективности моделей глубокого обучения, в том числе графовые нейронные сети (GNN) и крупные языковые модели (LLM), по сравнению с традиционными методами статического (SAST) и динамического (DAST) анализа. Показано, что GNN демонстрируют высокую точность в выявлении структурных ошибок безопасности памяти, в то время как LLM превосходят в семантическом анализе инъекционных уязвимостей, но при этом критически страдают от высокого уровня ложных срабатываний. Анализ сравнивается с ограничениями традиционного SAST и DAST. Определены фундаментальные ограничения ИИ-подходов, включая проблемы обобщаемости, уязвимость к состязательным атакам и риск генерации уязвимого кода.

Ключевые слова: уязвимости, искусственный интеллект, статистический и динамический анализы, сравнение.

Введение

Современный ландшафт разработки программного обеспечения (ПО) характеризуется беспрецедентным уровнем сложности, что неизбежно приводит к расширению поверхности атаки и росту числа критических дефектов. Глобальная статистика 2024 года отражает острую актуальность данной проблемы: общее количество публично раскрытых уязвимостей увеличилось на 25% за последний год, и эта тенденция к росту числа уязвимостей подчеркивает необходимость в более эффективных и масштабируемых методах защиты. Уязвимости, могут быть активированы или использованы злоумышленниками, приводя к нарушению безопасности [1].

Ключевым инструментом для стандартизации и приоритезации угроз является список MITRE Common Weakness Enumeration (CWE) Top 25 Most Dangerous Software Weaknesses, ежегодно публикуемый в качестве стратегического руководства. Список 2024 года классифицирует наиболее критические недостатки, которые MITRE идентифицирует как первопричины для 31,770 записей Common Vulnerabilities and

Exposures (CVE) в текущем наборе данных, используемом при составлении рейтинга. Определение корневых причин этих уязвимостей служит мощным рычагом для направления инвестиций и практик в области безопасности, позволяя устранять целые классы дефектов, таких как ошибки безопасности памяти или инъекции, еще на этапе архитектурного планирования [2].

Традиционно автоматизированное тестирование безопасности приложений (AST) опиралось на два фундаментальных, но взаимодополняющих подхода: статический анализ безопасности приложений (SAST) и динамическое тестирование безопасности приложений (DAST) [4, 13].

Традиционные SAST-инструменты представляют собой методы «белого ящика», которые анализируют исходный код без его исполнения. Их работа основана на предопределенных наборах правил и эвристик, предназначенных для выявления синтаксических ошибок кодирования. В свою очередь, традиционные DAST-инструменты функционируют как методы «черного ящика», анализируя поведение работающего приложения в реальном времени, что позволяет обнаружить runtime-слабости,

такие как ошибки конфигурации или недостаточные ограничения скорости.

Несмотря на их фундаментальную роль, традиционные методы имеют существенные ограничения. SAST-инструменты сталкиваются с проблемами масштабируемости и адаптивности. Их зависимость от фиксированных правил и комбинаторная сложность анализа кода ограничивают возможности, что приводит к высокому уровню ложных срабатываний (FPR). DAST, хотя и эффективен для обнаружения проблем среды исполнения, принципиально ограничен степенью покрытия исполняемых путей, поскольку его эффективность зависит от полноты используемых тестовых сценариев.

Масштаб и сложность уязвимостей, представленных в CWE Top 25, требуют смены парадигмы защиты. Для эффективной борьбы с современными угрозами необходим переход к системам, способным к дата-ориентированному обучению и выявлению новых паттернов.

Целью настоящего обзора является осуществление детального анализа эффективности систем обнаружения уязвимостей, основанных на искусственном интеллекте (ИИ), по отношению к традиционным методам SAST и DAST, с фокусом на уязвимостях CWE Top 25 2024. Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести сравнительную оценку традиционного SAST и моделей глубокого обучения (DL), акцентируя внимание на снижении FPR и семантическом моделировании кода.

2. Изучить методологии интеграции машинного обучения (ML) в DAST, включая интеллектуальный фаззинг и трассировку потоков данных.

3. На основе литературных данных определить классы CWE, где ИИ достигает максимального превосходства, обеспечивая более высокую точность и надежность обнаружения.

Основная часть

Common Weakness Enumeration (CWE) служит унифицированной, разработанной сообществом таксономией для

классификации слабостей программного и аппаратного обеспечения, которая в настоящее время включает более 600 категорий. Эта стандартизация является критически важной для обеспечения единообразия в управлении уязвимостями и проведения трендового анализа угроз.

Список CWE Top 25 2024 концентрируется на наиболее распространенных и критичных для эксплуатации слабостях, обеспечивая стратегическое руководство для разработчиков и аудиторов безопасности.

Традиционный SAST, основанный на предопределенных наборах правил, эффективен для выявления синтаксически очевидных ошибок, но испытывает трудности с адаптацией к новым векторам атак [3, 5].

Ключевым методологическим недостатком SAST является неспособность корректно учитывать контекст и семантику кода, что приводит к значительному высокому уровню ложных срабатываний (FPR) [4 – 5]. Подходы, полагающиеся на фиксированные эвристики, и комбинаторная сложность анализа, ограничивают возможности для обнаружения сложных, архитектурно обусловленных недостатков.

ИИ-подходы, основанные на глубоком обучении (DL), предлагают дата-ориентированную альтернативу, анализируя контекст, тенденции использования и исторические данные об уязвимостях для обнаружения сложных паттернов и активного снижения FPR [1, 9]. Ранние DL-подходы (RNN, LSTM, CNN) рассматривали код как линейную последовательность токенов [9, 12]. Хотя такие модели обладают определенными преимуществами в извлечении контекстуальной информации, они не способны эффективно захватить иерархические структуры программы, потоки выполнения, а также критические зависимости данных и контроля (Data/Control Dependencies).

Для преодоления этих ограничений в качестве доминирующей парадигмы утвердились графовые нейронные сети (GNN). GNN моделируют исходный код как графы (например, графы зависимостей

программ, PDG), что позволяет извлекать глубинную семантическую структуру. Использование гетерогенных GNN (HAGNN) позволяет учитывать разнородность типов узлов и агрегировать информацию через различные типы связей, точно захватывая структурные и контекстуальные свойства кода.

На крупных академических бенчмарках DL-модели демонстрируют производительность, значительно превышающую возможности традиционных инструментов. Например, передовой подход HAGNN достиг высокой точности (Accuracy): до 96.6% на наборе данных C и до 97.8% на наборе Java, превосходя лучшие базовые методы [1]. Эти высокие показатели достигнуты в специфических лабораторных условиях. На практике, при использовании GNN в разнородных промышленных средах, исследователи отмечают проблему обобщаемости (generalization), где средняя точность может варьироваться и быть существенно ниже, чем на тренировочных датасетах.

Сравнение с крупными языковыми моделями (LLM) показывает иную картину. LLM демонстрируют высокую чувствительность (Sensitivity или Recall), достигая 90%–100% обнаружения на лабораторных бенчмарках [7 – 8]. Однако это достигается ценой низкой точности (Precision), что выражается в критически высоком уровне ложных срабатываний (FPR). Анализ показывает, что традиционные SAST обладают низким FPR, но низкой общей скоростью обнаружения, тогда как LLM обеспечивают высокую скорость обнаружения, но требуют значительной дополнительной фильтрации из-за высокого FPR.

Высокая точность, достигаемая GNN, сталкивается с проблемой «черного ящика» [10 – 11]. Для оперативного устранения дефекта разработчикам требуется точное указание на корневую причину.

Для решения этой проблемы активно развиваются подходы Объясняемого ИИ (XAI). Новые методы, такие как контрафактическое объяснение (CFExplainer), преобразуют GNN-

предсказания в точные, действенные рекомендации. CFExplainer ищет минимальное возмущение во входном графе кода, которое бы изменило предсказание уязвимости. Это потенциально позволяет точно определить, какие конкретные узлы или ребра (зависимости) являются причиной дефекта. Однако, внедрение XAI в промышленные циклы остается активной областью исследований, требующей дальнейшего развития для достижения стабильности, надежности и полной интеграции с инструментами разработки.

DAST является незаменимым инструментом, поскольку оно выявляет уязвимости, проявляющиеся исключительно во время исполнения (runtime), такие как ошибки конфигурации или недостаточные runtime-ограничения [4, 17].

Ключевым методологическим недостатком DAST является его прямая зависимость от покрытия кода (code coverage). Несмотря на то, что современные DAST-решения используют интеллектуальные краулеры и эвристики, их результативность полностью определяется тем, насколько полно тестовые сценарии могут инициировать все потенциально уязвимые пути исполнения. Это фундаментальное ограничение: покрытие любого набора тестов конечно и ограничено исполненными сценариями. Следовательно, DAST часто демонстрирует высокий уровень ложноотрицательных результатов (False Negative Rate, FNR) для глубоко скрытых или редко используемых функций.

Исторически DAST-инструменты были разработаны для сканирования традиционных, серверно-ориентированных приложений. Однако их эффективность резко снижается в современных архитектурах, таких как Одностраничные Приложения (SPA), где навигационная логика и значительная часть функционала встроены в клиентский код JavaScript. Традиционные динамические краулеры не способны адекватно обнаруживать и обходить все пути исполнения, поскольку эти пути не генерируются сервером. В результате DAST пропускает класс уязвимостей, часто связанный с клиентской логикой. Для обхода

этих ограничений развиваются гибридные методы, основанные на статическом анализе. Например, некоторые подходы используют статический анализ JavaScript-бандлов для извлечения навигационных графов, что позволяет искусственно "направлять" сканирование и получать более полное покрытие, тем самым компенсируя ограниченность традиционных динамических краулеров.

Искусственный интеллект радикально улучшает DAST, предоставляя интеллектуальные механизмы, направленные на повышение покрытия и эффективности тестирования.

ML-Управляемый Фаззинг (Fuzz Testing): машинное обучение применяется для оптимизации процесса фаззинга, что приводит к значительному увеличению покрытия и эффективности обнаружения уязвимостей. ML-модели, используя такие техники, как Генеративно-состязательные сети (WGAN), могут анализировать семантические сходства между протоколами или структурой данных, что позволяет им генерировать высокоэффективные тестовые случаи, нацеленные на максимальное проникновение в новые, ранее неиспользовавшиеся пути кода [15].

Продвинутое Трассирование Поточных Данных (Taint Tracking): в области динамического анализа потоков данных (DTA) ИИ используется для обнаружения неявных (implicit) потоков данных, где традиционные, правило-основанные механизмы DTA могут ошибочно недооценивать степень заражения (undertainting) [16]. Такие неявные потоки, в которых путь управления полностью определяет значение входного значения, часто являются причиной уязвимостей, которые DAST без интеллектуальной поддержки пропускает.

Наиболее перспективное развитие лежит в гибридных моделях, таких как интерактивное тестирование безопасности приложений (IAST), усиленное ИИ, которое сочетает преимущества SAST и DAST. ИИ-компоненты в IAST используют данные об исполнении, собранные DAST, для приоритизации результатов статического

анализа (SAST/GNN) [13, 18]. Это обеспечивает более высокую точность и способствует снижению общего FPR, чем любой из методов по отдельности.

Сравнительная характеристика традиционных и ИИ-управляемых методов обнаружения уязвимостей выявляет существенные различия по ключевым параметрам. Традиционный SAST (Правило-Основанный) демонстрирует низкую скорость обнаружения, но обладает относительно низким FPR. Традиционный DAST (Исполнение) имеет высокий FNR из-за ограничения покрытия кода. В отличие от них, системы обнаружения уязвимостей на базе AI/ML (включающие GNN и LLM) обладают высокой масштабируемостью. LLM, обладая высокой чувствительностью (Recall), компенсируют низкую точность (Precision) SAST, но при этом сами страдают от высокого FPR, требуя фильтрации. Гибридные IAST-модели, управляемые ИИ, стремятся объединить сильные стороны, используя динамические данные для повышения точности статических предсказаний, тем самым снижая общий FPR/FNR.

Одним из наиболее значимых преимуществ ИИ перед традиционным SAST является его высокая эффективность в обнаружении уязвимостей безопасности памяти, таких как CWE-787 (Out-of-Bounds Write) и CWE-119 [9, 12]. Эти ошибки требуют анализа сложных, долгосрочных зависимостей потоков данных и управления в межпроцедурном контексте.

GNN-модели, напротив, изначально спроектированы для обработки сложных структур данных. Используя гетерогенные графы, они способны захватывать эти критические зависимости наиболее точно. Количественные результаты подтверждают это превосходство: GNN-подходы демонстрируют высокую точность, что делает их оптимальным решением для обнаружения этих системно критических дефектов.

Для инъекционных уязвимостей, таких как CWE-89 (SQL Injection) и CWE-94 (Code Injection), ИИ демонстрирует преимущество,

связанное с его способностью к контекстному и семантическому анализу [7].

Там, где традиционные анализаторы ищут простые паттерны, ИИ, в особенности Large Language Models (LLM) и семантические анализаторы, могут идентифицировать сложные, многоступенчатые векторы инъекций. Они способны выявить некорректное использование функций санитарной обработки в различных контекстах или идентифицировать, когда данные проходят через несколько уровней преобразования, прежде чем попасть в интерпретатор. Кроме того, тот факт, что в существующие категории CWE, включая CWE-89 и CWE-94, были внесены специфические примеры, связанные с ИИ-системами, подчеркивает критическую роль ИИ в обнаружении этих сложных, обновленных векторов атак.

Наиболее значимое качественное превосходство ИИ заключается в его способности обнаруживать логические и архитектурные ошибки, которые традиционно игнорируются SAST, поскольку они не нарушают синтаксис. Примером служит CWE-862 (Missing Authorization).

ML-классификаторы, обученные на больших массивах данных безопасного и уязвимого кода, способны выявлять аномалии в высокоуровневых структурах функций [1]. Они могут обнаружить, что критический блок кода, который по всем семантическим и контекстуальным признакам должен включать проверку авторизации, не соответствует ожидаемому «безопасному» шаблону. Таким образом, ИИ-модели эффективно определяют нарушения логики безопасности, действуя как высокоскоростной, обученный аудитор, что недоступно для традиционных правил-основанных систем.

Уязвимости, входящие в CWE Top 25, охватывают широкий спектр дефектов, требующих как структурного, так и семантического понимания. Ошибки безопасности памяти (CWE-787) являются структурными по своей сути и оптимально обнаруживаются GNN. Напротив, инъекции (CWE-89) и логические ошибки (CWE-862) требуют глубокого семантического и

контекстуального анализа, в чем сильны LLM.

Достижение максимальной эффективности требует разработки мультимодальных систем, которые одновременно обрабатывают код как граф (для точного потокового анализа) и как естественный язык (для семантического и логического анализа). Объединение этих разнородных представлений требует разработки сложных мультимодальных архитектур. Это сопряжено с высокими вычислительными затратами и необходимостью согласования разнородных представлений, что является серьезным вызовом для масштабируемости в реальных DevSecOps-конвейерах.

Определение специфической эффективности ИИ в обнаружении критических CWE из списка Top 25 позволяет соотнести тип уязвимости с наиболее адекватным методологическим подходом. Для структурных ошибок, таких как CWE-787 (Out-of-Bounds Write), оптимальными являются Графовые Нейронные Сети (GNN). Уязвимости класса CWE-89 (SQL Injection) требуют LLM и семантического анализа. Для CWE-862 (Missing Authorization), представляющей собой логическую ошибку безопасности, наилучшими являются ML-Классификаторы и LLM. Наконец, для уязвимостей исполнения, таких как CWE-78 (OS Command Injection), наиболее эффективен AI-Управляемый Фаззинг в сочетании с Динамическим Анализом Поточков Данных (DTA).

Заключение

Проведенный анализ свидетельствует о том, что технологии Искусственного Интеллекта представляют собой эволюционный сдвиг в области автоматизированного тестирования безопасности приложений, предлагая решения, способные преодолеть фундаментальные ограничения традиционных методов. Традиционные методы SAST и DAST сохраняют свою роль как базовые инструменты, но их ограничения в масштабируемости, адаптивности и

точности становятся критичными в контексте сложных уязвимостей CWE Top 25.

DL-модели на основе GNN продемонстрировали выдающуюся прогностическую точность на академических бенчмарках [9, 12], превосходящую традиционные SAST за счет глубокого структурного и семантического анализа, что делает их оптимальным решением для обнаружения ошибок безопасности памяти. Интеграция ИИ в DAST критически повышает покрытие кода и эффективность обнаружения runtime-уязвимостей, а также позволяет интеллектуально исследовать сложные логические недостатки.

Несмотря на значительные успехи, массовое промышленное внедрение ИИ ограничивается рядом ключевых факторов. Во-первых, эффективность моделей глубокого обучения напрямую зависит от наличия обширных, высококачественных и аннотированных наборов данных. Дефицит, а также шум и несбалансированность существующих данных, представляют серьезное препятствие. Во-вторых, критическим вызовом остается высокий уровень ложных срабатываний (FPR) у LLM, несмотря на их высокую чувствительность, продемонстрированную на лабораторных бенчмарках [1, 7].

Более глубокие методологические ограничения ИИ включают:

1. Генерация уязвимого кода. Помимо проблем с обнаружением, LLM представляют риск как инструмент, поскольку могут генерировать код с уязвимостями по умолчанию. В зависимости от типа уязвимости (например, Buffer Overflow), частота генерации небезопасного кода может достигать 78.8%. Даже при использовании прямых запросов на «безопасный код» доля уязвимых ответов остается высокой, что требует ручного аудита.

2. Проблема обобщаемости (Generalization). Высокая точность, достигнутая на одном бенчмарке (например, 96.6% Accuracy GNN), может не сохраняться в реальных, разнородных кодовых базах, где фактическая производительность значительно варьируется.

3. Уязвимость к состязательным атакам (Adversarial Attacks). Небольшие, незаметные для человека изменения в коде могут обмануть ИИ-модель, что ставит под сомнение ее надежность в среде, где предполагается враждебное воздействие.

Будущее автоматизированного обнаружения уязвимостей сосредоточено в двух ключевых направлениях.

Первое — разработка гибридных и ансамблевых моделей, таких как IAST-системы, объединяющих сильные стороны традиционного SAST (надежный низкий FPR) с прогностической мощью GNN и LLM, что позволит смягчить недостатки каждого подхода.

Второе — развитие объясняемого ИИ (XAI). Технологии контрафактического объяснения, такие как CFExplainer, имеют решающее значение для преобразования GNN-предсказаний из «черного ящика» в точные, действенные рекомендации для разработчиков. XAI является активной областью исследований, критически важным шагом для повышения доверия к автоматизированным системам. Кроме того, будущие исследования должны учитывать более широкое пересечение проблем безопасности, включая интеграцию с новыми стандартами, такими как MITRE 2025 Most Important Hardware Weaknesses (MIHW), что требует разработки комплексных решений для оценки безопасности как ПО, так и аппаратных компонентов [19].

Список литературы

1. AI efficacy memory safety injection vulnerabilities scientific review // ArXiv : электрон. науч. архив. URL: <https://arxiv.org/abs/2508.20866v1>. (Дата обращения: 28.09.2025).
2. 2024 Common Weakness Enumeration (CWE™) Top 25 Most Dangerous Software Weaknesses List // MITRE. URL: <https://cwe.mitre.org/top25/>. (Дата обращения: 28.09.2025).
3. [Vulnerability detection using static analysis in code reviews] // ArXiv : электрон. науч. архив. URL: <https://arxiv.org/abs/2407.16235>. (Дата обращения: 28.09.2025).

4. Yildiz O. Mastering DAST vs. SAST: An ultra-extensive guide to application security testing // Medium. 2023. URL: <https://medium.com/@okanyildiz1994/mastering-dast-vs-sast-an-ultra-extensive-guide-to-application-security-testing-aadcc6c26478>. (Дата обращения: 28.09.2025).
5. AI-powered Static Application Security Testing // Akto. URL: <https://www.akto.io/learn/ai-powered-static-application-security-testing>. (Дата обращения: 28.09.2025).
6. Common Weakness Enumeration // Wikipedia. URL: https://en.wikipedia.org/wiki/Common_Weakness_Enumeration. (Дата обращения: 28.09.2025).
7. New Content for Your Most Pressing Emerging Vulnerabilities: AI/LLM CWE Top 25 // Security Journey. URL: <https://www.securityjourney.com/post/new-content-for-your-most-pressing-emerging-vulnerabilities-ai/llm-cwe-top-25>. (Дата обращения: 28.09.2025).
8. Leveraging hardened cybersecurity frameworks for AI security // Cisco Blogs. 2024. URL: <https://blogs.cisco.com/security/leveraging-hardened-cybersecurity-frameworks-for-ai-security>. (Дата обращения: 28.09.2025).
9. [GNN for code representation and vulnerability detection] // ArXiv : электрон. науч. архив. URL: <https://arxiv.org/html/2404.14719v1>. (Дата обращения: 28.09.2025).
10. CFExpainer: A Counterfactual Explainer for GNN-based Vulnerability Detection // ArXiv : электрон. науч. архив. URL: <https://arxiv.org/abs/2404.15687>. (Дата обращения: 28.09.2025).
11. CFExpainer: A Counterfactual Explainer for GNN-based Vulnerability Detection // ArXiv : электрон. науч. архив. URL: <https://arxiv.org/abs/2404.15687>. (Дата обращения: 28.09.2025).
12. [Heterogeneous Attention GNN for vulnerability prediction] // ArXiv : электрон. науч. архив. URL: <https://arxiv.org/abs/2502.16835v1>. (Дата обращения: 28.09.2025).
13. Static and Dynamic Application Security Testing (SAST) and DAST Integration for Robust Secure Coding Practices // ResearchGate : электрон. науч. ресурс. URL: https://www.researchgate.net/publication/393516606_Static_and_Dynamic_Application_Security_Testing_SAST_and_DAST_Integration_for_Robust_Secure_Coding_Practices. (Дата обращения: 28.09.2025).
14. // ArXiv : электрон. науч. архив. URL: <https://arxiv.org/abs/2407.16235>. (Дата обращения: 28.09.2025).
15. // ResearchGate : электрон. науч. ресурс. URL: https://www.researchgate.net/publication/376846314_Machine_Learning-based_Fuzz_Testing_Techniques_A_Survey. (Дата обращения: 28.09.2025).
16. Online Taint Propagation Using DTA++ rules // BitBlaze. URL: <http://bitblaze.cs.berkeley.edu/papers/dta%2B%2B-ndss11.pdf>. (Дата обращения: 28.09.2025).
17. A Comparative Analysis and Benchmarking of Dynamic Application Security Testing (DAST) Tools // ResearchGate : электрон. науч. ресурс. URL: https://www.researchgate.net/publication/385500967_A_Comparative_Analysis_and_Benchmarking_of_Dynamic_Application_Security_Testing_DAST_Tools. (Дата обращения: 28.09.2025).
18. Static and Dynamic Application Security Testing (SAST) and DAST Integration for Robust Secure Coding Practices // ResearchGate : электрон. науч. ресурс. URL: https://www.researchgate.net/publication/393516606_Static_and_Dynamic_Application_Security_Testing_SAST_and_DAST_Integration_for_Robust_Secure_Coding_Practices. (Дата обращения: 28.09.2025).
19. Маслова, М. А. Метод статистического анализа локальной сети и выявления компьютерных атак на пакетных выборках оптимальной длины / М. А. Маслова, А. А. Белоус // Информация и безопасность. 2024. Т. 27, № 4. С. 561-570.

Севастопольский государственный университет
Sevastopol State University

Поступила в редакцию

Информация об авторе

Захаров Алексей Павлович – студент ФГАОУ ВО «Севастопольский государственный университет» кафедры «Информационная безопасность», e-mail: alechazarov@mail.ru; ул. Университетская, д. 33, г. Севастополь, 299053, Россия.

Маслова Мария Александровна – доцент кафедры «Информационная безопасность», Севастопольский государственный университет, e-mail: mashechka-81@mail.ru; ул. Университетская, д. 33, г. Севастополь, 299053, Россия.

**THE USE OF ARTIFICIAL INTELLIGENCE IN THE SEARCH FOR CODE
VULNERABILITIES AND COMPARATIVE ANALYSIS OF VARIOUS TESTING
METHODS**

A.P. Zakharov, M.A. Maslova

The article provides an analytical overview of the possibilities of using artificial intelligence technologies to detect critical vulnerabilities included in the MITRE CWE Top 25 list of 2024. The study contains a synthesis and critical comparison of literature data on the effectiveness of deep learning models, including graph neural networks (GNN) and large language models (LLM), compared with traditional methods of static (SAST) and dynamic (DAST) analysis. It is shown that GNNs demonstrate high accuracy in detecting structural memory security errors, while LLMs excel in the semantic analysis of injection vulnerabilities, but at the same time critically suffer from a high level of false positives. The analysis is compared with the limitations of traditional SAST and DAST. The fundamental limitations of AI approaches are identified, including generalizability issues, vulnerability to adversarial attacks, and the risk of vulnerable code generation.

Keywords: vulnerabilities, artificial intelligence, statistical and dynamic analyses, comparison.

Submitted

Information about the author

Aleksey P. Zakharov – Student of FGAOU VO 'Sevastopol State University', Department of 'Information Security', e-mail: alechazarov@mail.ru; 33 Universitetskaya St., Sevastopol, 299053, Russia.

Maria A. Maslova – Associate Professor of the Department of Information Security, Sevastopol State University, e-mail: mashechka-81@mail.ru; 33 Universitetskaya str., Sevastopol, 299053, Russia.